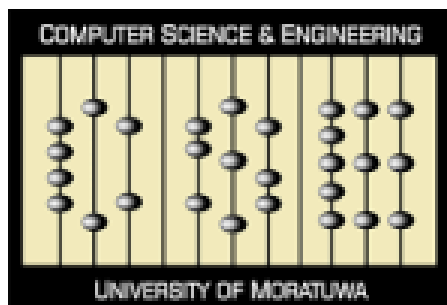




Improving Blockchain Scalability: Throughput Enhancement in ZK-Rollups

Student One¹, Student Two, Student Three, Student Four

Department of Computer Science & Engineering, University of Moratuwa, Sri Lanka

1. ABSTRACT

- Problem:** Blockchain networks (e.g., Ethereum) suffer from low throughput and high transaction fees.
- Solution:** Zero-Knowledge (ZK) Rollups batch transactions off-chain and submit a single cryptographic proof to the main chain.
- Contribution:** We propose a novel, optimized ZK-Rollup prototype implemented in Rust that increases transaction throughput by parallelizing proof generation and optimizing the Merkle tree state updates.
- Result:** Achieved a 3x throughput enhancement compared to baseline sequential implementations, drastically reducing off-chain computational overhead.

2. OBJECTIVES

- High Throughput:** Design a parallelized proof generation pipeline.
- Low Latency:** Optimize state transitions in the underlying Merkle structures.
- Cost-Efficiency:** Minimize gas costs on the Layer-1 verification contract.
- Security:** Ensure rigorous cryptographic soundness utilizing SNARKs.

3. BACKGROUND

- Layer-1 Limitations:** Sequential execution and consensus bottlenecks limit Ethereum to ≈ 15 TPS.
- ZK-Rollups:** An L2 scaling solution that bundles hundreds of transfers into a single transaction.
- Current Bottleneck:** Generating the ZK-SNARK proofs is computationally intensive and currently represents the primary bottleneck for rollup sequencers.

6. CONCLUSIONS

- Scalability Achieved:** Parallel proof generation effectively shifts the bottleneck away from computation, maximizing ZK-Rollup throughput.
- Efficiency:** The Rust-based prover network demonstrated near-linear scaling up to 16 cores.
- Future Work:** Integrating hardware acceleration (GPUs/FPGAs) to further reduce proof latency and evaluating recursive SNARK limits.

7. REFERENCES

4. ARCHITECTURE

- Sequencer Node:** Receives user transactions, validates signatures, and batches them.
- Prover Network (Rust):** Distributed worker nodes that generate SNARK proofs in parallel for distinct transaction chunks.
- Aggregator:** Combines individual proofs into a single recursive proof.
- Smart Contract (Solidity):** Verifies the final aggregated ZK-proof on the Ethereum mainnet.

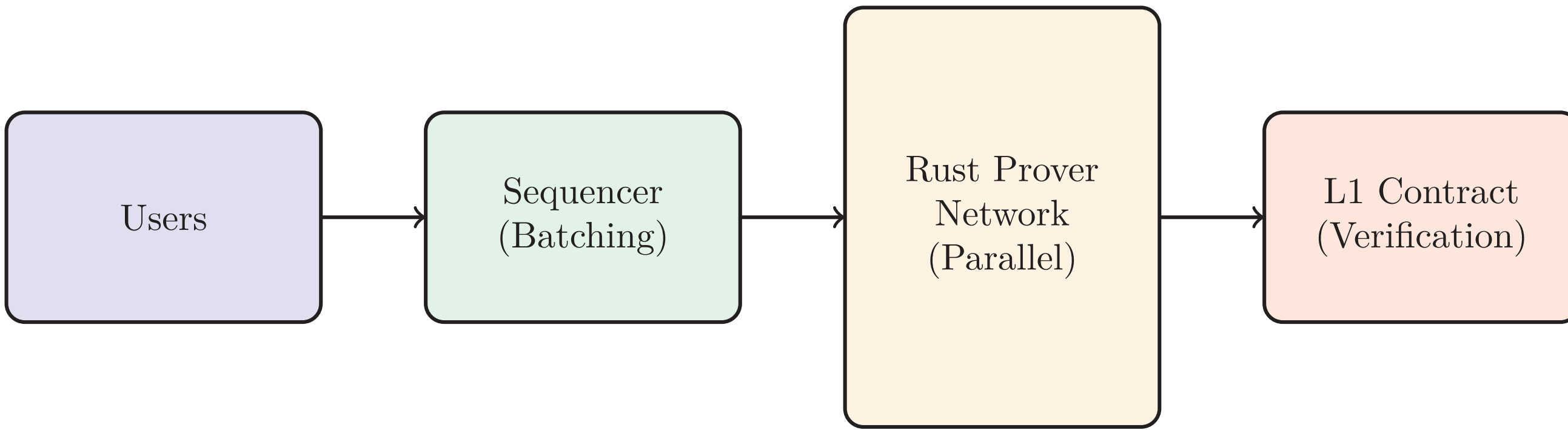


Figure 1: System Architecture Diagram

5. IMPLEMENTATION

- Proving System:** Implemented using the `bellman` and `halo2` Rust crates.
- Parallelization:** Utilized `rayon` for multi-threaded state tree updates and proof generation.
- Testing:** Benchmarked on an AWS EC2 instance (32 cores, 64GB RAM) simulating a high-load sequencer environment.

Implementation	Batch Size	Proof Time (s)	TPS
Baseline (Sequential)	1000	45.2	22.1
Ours (4 Threads)	1000	14.8	67.5
Ours (16 Threads)	1000	4.5	222.2
Ours (32 Threads)	1000	2.8	357.1

Table 1: Throughput Comparison (Rust Implementation)